Ders 9 Çalışma Özeti

Ders 9 Çalışma Özeti

Genel Konular

- Code smell: Large Class (aşırı büyük sınıf)
  - Sınıfın büyük olması büyük olasılıkla uyumunun düşük olduğunu, yani birbirine yakın olmayan, kimi zaman ilgisiz sorumlulukların aynı sınıfa atandığını gösterir. Üye alan sayısı fazladır; asıl sorun da üye metotların çok satır içermesidir. Önerilen uzunluğun on katı olan dört beş metot bile bir sınıfı aşırı büyük yapar.
  - Çözümler: Extract Class, Extract Subclass ve (sunum ile iş mantığı karışmışsa) Duplicate Observed Data.
  - Sunum katmanı ile iş mantığı katmanı ayrılmazsa her iş sınıfı kendini kullanıcıya gösteren üyeleri (formlar, diyalog kutuları vb.) de taşır ve kod en az iki katına çıkar.
- Refactoring: Extract Class (sınıf çıkartma) (ekrandan, 00:10:00)
  - Birbiriyle ilgisiz iki iş yapan bir sınıfı bu işlere göre ikiye ayırmaktır: ikinci işle ilgili üyeler seçilip yeni bir sınıfa taşınır. Birçok kod kusurunu düzeltir.
  - Örnek: `Person` (`name`, `officeAreaCode`, `officeNumber`, `getTelephoneNumber`) → `Person` (`name`, `getTelephoneNumber`) ve `officeTelephone` ilişkisiyle (çokluk 1) bağlı `Telephone Number` (`areaCode`, `number`, `getTelephoneNumber`).
  - Kişinin iş, ev, cep telefonları için her numaraya iki alan yerine tek bir nesne alanı tutulur; numara yoksa alan `null` bırakılır, "0" gibi anlamsız değer atamak gerekmez. Telekom sistemindeki şebeke içi/dışı karşılaştırma gibi telefonla ilgili metotlar da yeni sınıfa gider, uyum yükselir.
- Uyumun üye alan erişim deseniyle hesaplanması (ekrandan, 00:16:40)
  - `Person` sınıfında 4 üye alan (`name`, `id`, `officePhoneCode`, `officePhoneNr`) ve 6 metot (m1–m6) vardır; metottan alana çizilen her ok bir erişimdir. Gerçekleşen erişim 10, olası en fazla erişim $6 \times 4 = 24$; uyum $10/24 \approx 0{,}42$.
  - Extract Class sonrası: birinci sınıfta 4 metot ve 2 alan, olası 8 erişimden 6'sı vardır → $6/8 = 0{,}75$. İkinci sınıfta 2 metot ve 2 alan, 4 erişimin 4'ü de vardır → $4/4 = 1$ (%100).
- Refactoring: Extract Subclass (alt sınıf çıkartma) (ekrandan, 00:20:00)
  - Bir sınıfın bazı üyeleri her zaman kullanılmıyorsa, bu üyelerden yeni bir alt sınıf oluşturmak daha doğrudur.
  - Örnek: `Job Item` (`getTotalPrice`, `getUnitPrice`, `getEmployee`). Bazı işlerde birim fiyata çalışanın emeği de eklenir. `Labor Item` alt sınıfı oluşturulur, `getEmployee` ona taşınır, `getUnitPrice` orada yeniden tanımlanır (override).
- Refactoring: Duplicate Observed Data
  - Uygulama alanı verisinin GUI sınıfında saklanması yanlıştır ama yapılabilir; canlıya çıkmış kodda geriye doğru uyumluluk korunarak düzeltilir.
  - Örnek: `IntervalWindow` hem veri girişi alanlarını, hem odak kazanma/kaybetme gibi GUI olaylarını, hem de uzunluk hesabı gibi iş mantığını içerir. İş mantığı yeni bir `Interval` sınıfına çıkarılır, GUI sınıfı silinmez; ikisi arasına Observer kalıbı kurulur.
  - Hoca farkı şöyle özetledi: bir varlık değişince değişimi gösteren bir GUI söz konusuysa (örneğin borsa endeksi izleme) yapılan bölme işlemi Duplicate Observed Data'dır; öyle değilse yalnız Extract Class yapılmıştır.
- Code smell: Divergent Change ve Shotgun Surgery
  - İkisinin özü aynıdır: bir hata ya da gereksinim değişikliği için birden fazla yere dokunmak gerekir.
  - Değişiklikler aynı sınıfın birçok metodunda yapılıyorsa Divergent Change; birçok sınıfın az sayıdaki (bir iki, en fazla üç) metodunda yapılıyorsa Shotgun Surgery denir.

  - Shotgun Surgery benzetmesi: av tüfeği (çifte) kartuşundaki saçmalar ateşlenince yayılır; değişiklikler de tasarımın üzerine saçılmıştır.
  - Çözümler: Extract Class (dağınık metotlar birlikte anlam taşıyorsa onlardan yeni sınıf çıkarılır) ya da mevcut sınıflardan birine toplamak: Move Field, Move Method.
- Refactoring: Move Method / Move Field
  - Bir metot kendi sınıfından çok başka bir sınıftan çağrılıyorsa yanlış yerde tanımlanmıştır; ona en çok ihtiyaç duyan sınıfa taşınır.
  - Eski sınıftan tamamen silinmez, çünkü o sınıfın arayüzü değişir. Eski yerde yalnız imza kalır ve çağrıyı yeni yere iletir (delegation; bir tür proxy gibi).
  - Başka kusurları düzeltir, Extract Class gibi daha karmaşık eylemlerin ilk adımı olarak da kullanılır.
- Code smell: Primitive Obsession (ilkellere takıntı)
  - Yapısal programlamadan gelenlerin ilkel tipleri aşırı kullanma eğilimidir. Çözüm eylemlerinin hepsi "Replace" ile başlar: Replace Data Value with Object, Replace Type Code with Class / Subclasses / State-Strategy, Replace Array with Object.
  - Önceki haftaların Extract Class ve Introduce Parameter Object eylemleri baştan kullanılsaydı bu kusur hiç oluşmayabilirdi.
- Refactoring: Replace Data Value with Object (ekrandan, 00:40:00)
  - Bir üye alan, hele onunla ilgili davranışlar varsa, bir sınıfla temsil edilmelidir. Örnek: `Order` içinde `customer: String` → `Order`, çokluğu 1 olan bir ilişkiyle `Customer` (`name: String`) sınıfına bağlanır.
  - Ad tekil bir tanımlayıcı değildir: aynı ad ve soyadlı farklı müşteriler olabilir (hoca kendi adının başka bir çocukta da çıktığını örnek verdi). Bir müşterinin tüm siparişlerini aramak gibi işlemler çoğaldıkça `Customer` ayrılmazsa `Order` aşırı büyür.
- Refactoring: Replace Type Code with Class
  - Örnek: kan grubu `int` olarak tutulur; O, A, B, AB için public final static sabitler (0, 1, 2, 3) tanımlanır. Bunun yerine bir `BloodGroup` sınıfı oluşturulur, `Person`'daki üye alanın türü `BloodGroup` olur.
  - Dil destekliyorsa `enum` kullanmak da aynı sonucu verir. Java'da enum'lar sınıf gibi işlenir; her değeri o türden bir nesnedir ve kurucu, alan ve metotları olabilir.
- Refactoring: Replace Type Code with Subclasses (ekrandan, 00:50:00)
  - Bir sınıfın davranışını belirleyen ilkel tip kodu yerine kalıtım ve çok biçimlilik kullanılır. Örnek: `Employee` (`ENGINEER: int`, `SALESMAN: int`, `type: int`) → `Employee` ve alt sınıfları `Engineer`, `Salesman`.
  - Tip kodu kalırsa maaş ve özlük hakkı metotlarının hepsinde switch-case çıkar; bu Long Method'a, onlar da birleşip Large Class'a yol açar. Türü olmayan çalışan olamıyorsa `Employee` soyut yapılır.
- Refactoring: Replace Type Code with State/Strategy (ekrandan, 00:47:20)
  - Kalıtım bir nedenle kullanılamıyorsa: `Employee` → (1) `Employee Type`; altında `Engineer`, `Salesman`. `Employee Type` arayüz ya da soyut sınıf olabilir.
  - Alt sınıf çözümünde nesnenin türü sonradan değiştirilemez; mühendisi satış elemanı yapmak için verisi kopyalanıp yeni nesne oluşturulmalı, eskisi silinmelidir (copy constructor ya da `Company` sınıfında `promote`/`changeType` gibi zahmetli bir metot). Strateji çözümünde yalnız `Employee Type` üyesine yeni nesne atanır, nesnenin kimliği değişmez.
  - Tür çalışma sırasında değişebiliyorsa (terfi, kadro değişimi) yapı State kalıbıdır; yalnız hesap türe göre farklıysa Strategy kalıbıdır.
- Refactoring: Replace Array with Object
  - Örnek: ligdeki takımların adları, puanları ve averajları üç ayrı dizide tutulur, aynı indis aynı takımı gösterir. Bu hataya açıktır (hoca canlı yazarken bir hata yaptı). Bunun yerine `Takım` sınıfı oluşturulur ve tek bir nesne dizisi kullanılır.
- Code smell: Feature Envy (özellik kıskançlığı) (ekrandan, 01:10:00)
  - Bir metot ait olduğu sınıfın üyelerinden çok başka sınıfların üyelerine erişiyorsa ya da başka sınıflardan daha çok çağrılıyorsa. "Çok", o koddaki ortalamanın (veya %20 gibi bir eşiğin) üzerindeki sayıdır; eşik daha önceki kaliteli projelerden, literatürden ya da aynı projenin ortalamasından alınabilir.
  - Sorumlulukların doğru atanmadığının göstergesi olabilir. Sıfır bağlaşım olamaz; hiçbir sınıf yalnız kendi metotlarını kullanmaz, öyle olsaydı uyum olumsuz etkilenirdi. Strategy ve Visitor kalıpları özellik kıskançlığı sayılmaz.
  - Çözüm: Move Method (eski yerde tek satırlık delegation kalır) ya da kıskançlık me-

todun bir kısmındaysa Extract Method ile o kısım çıkarılıp ilgili sınıfa taşınır.

- Fowler'ın video kiralama örneğinin adım adım refactoring'i (Fowler, Refactoring kitabı, 1. bölüm)
  - Kod Eclipse'te `fowler_00` … `fowler_11_alt` paketlerinde, testler `testing` altında aynı paket adlarıyla tutulur. Her adımda paket kopyalanır, değişiklik yeni pakette yapılır, test kodu değiştirilmeden yeniden çalıştırılır.
  - İlk tespitler: `Customer.statement()` çok uzundur; uzunluğu artıran switch-case tip kodundan kaynaklanır; geçici değişkenler metodu uzatır; `Customer`'ın `Rental` üzerinden `Movie`'ye erişmesi dolaylı ilişkinin aşırı kullanımı ya da Feature Envy olabilir.
  - Adım 1 (`fowler_01`, ekrandan, 01:20:00): switch bloğu Extract Method ile `Customer` içinde private `amountFor(Rental each)` metoduna çıkarılır.
  - Adım 2: anlaşılabilirlik için yeniden adlandırma (`each` → `aRental`, `thisAmount` → `result`); Eclipse'te Refactor → Rename ile.
  - Adım 3 (`fowler_03`, ekrandan, 01:40:00): `amountFor` `Customer`'ın hiçbir üyesine erişmez; Move Method ile `Rental`'a taşınır ve adı `getCharge()` olur.
  - Adım 4 (`fowler_04`, ekrandan, 01:50:00): `thisAmount` geçici değişkeni yalnız bir kez atanıp hiç değişmediği için kaldırılır, yerine `aRental.getCharge()` doğrudan çağrılır (Replace Temp with Query).
  - Adım 5 (`fowler_05`, ekrandan, 01:55:30): puan hesabı Extract Method + Move Method ile `Rental.getBonus()` olur; `frequentRenterPoints` adı `bonus` yapılır; mantık sadeleşir: yeni film 1 günden uzun kiralandıysa 2, değilse 1 puan.
  - Adım 6 (`fowler_06`, ekrandan, 02:17:20): `totalAmount` ve `bonus` toplamları `statement()`'tan çıkarılır; `Customer`'a `getTotalCharge()` ve `getTotalBonus()` eklenir (Replace Temp with Query).
  - Adım 7 (`fowler_07`, ekrandan, 02:30:00): switch-case `Rental`'dan `Movie.getCharge(int daysRented)`'e taşınır; `Rental.getCharge()` tek satırlık iletim olur. Yeni film türü eklenince değişiklik yalnız `Movie` ile sınırlı kalır.
  - Adım 8: aynısı bonus için: `Movie.getBonus(int daysRented)`.
  - Adım 9 (`fowler_09`, ekrandan, 02:40:00): `Customer`'ın `Movie`'ye doğrudan tek erişimi (film adı) kaldırılır; `Rental.getMovieTitle()` eklenir. Customer → Rental → Movie katmanlaması sağlanır.
  - Kalan iş: `statement()` içindeki footer kısmı ayrı bir metoda (add footer) çıkarılabilir; hoca bunu sonraya bıraktı.
  - Alternatif 1 (Replace Type Code with Subclasses): soyut `Movie` ve `NewRelease`, `Regular`, `Childrens` alt sınıfları. Ama `Movie` nesnesi oluşturulamayacağı için dış arayüz ve test kodu değişir, geriye doğru uyumluluk bozulur.
  - Alternatif 2 (`fowler_10_alt`, Replace Type Code with Strategy, ekrandan, 02:46:40): `Movie` (`-_price: Price`) → (1) soyut `Price` (`getCharge(daysRented: int): double`, `getBonus(daysRented: int): int`); alt sınıflar `PriceNewRelease`, `PriceRegular`, `PriceChildrens`. Test kodu değişmez; hoca bunu tercih etti.
- Kara kutu ve beyaz kutu sınaması
  - Kara kutu: iç işleyiş bilinmez; geçerli girdilere doğru çıktı verilip verilmediğine bakılır. Girdi grupları öngörülebiliyorsa her gruptan bir girdi en doyurucu testtir (örnek: yaş gruplarına göre olması gereken kan basıncı).
  - Beyaz kutu: iç işleyiş bilinir; bütün dallardan ve döngü sınırlarından geçen girdilerle en yüksek kod kapsamı (code coverage) hedeflenir. Daha kapsamlıdır ama daha çok iş gerektirir.
  - Genelde kritik ve karmaşık parçalar için beyaz kutu, basit parçalar için kara kutu yeterlidir. Örnekte `Rental.getCharge()` için yeni test yazılmadı; mevcut testlerle kara kutu yaklaşımıyla sürdürüldü.
- Anti patterns (karşıt kalıplar) (ekrandan, 02:50:00)
  - Tasarım kalıbı gibi görünür ama öyle değildir; yazılım geliştirme sürecinde tekrar eden yanlışları ifade eder. Bilmek, ciddi problemleri önceden tahmin etmeyi ve tedbir almayı kolaylaştırır.
  - Kod kusurları bir metot, sınıf ya da en fazla bir iki sınıf düzeyindedir ve gerçekleme düzeyine daha yakındır; karşıt kalıplar birden fazla sınıfa yayılır, tasarım düzeyine yakındır ve yazılım yaşam döngüsünün diğer evreleriyle de ilişkilidir.
  - Üç grup: yazılım geliştirme, yazılım mimarisi ve yazılım proje yönetimi karşıt kalıpları. Hoca konunun müfredatta olmadığını, yalnız varlığının bilinmesini istedi.
- İkinci ara sınavın soruları üzerinden tasarım kalıbı tartışması (ekrandan, 03:00:00)

  - Strategy ile State farkı, Observer ile Mediator karşılaştırması, Chain of Responsibility, bağlaşımı azaltan kalıplar ve switch-case'ten kurtaran kalıplar konuşuldu. Ayrıntısı genişletilmiş notun "Örnek Sorular" bölümündedir.

Hocanın Özellikle Vurguladığı Kısımlar

- Kod kusurlarını ve refactoring eylemlerini özgün İngilizce adlarıyla bilmek ve hangi kusurun hangi eylemle düzeltildiğini eylemin yaptığı işten çıkarabilmek
  - Dersin kazandırması gereken temel beceri budur.
- Switch-case ya da if-else-if zinciri yazıyorsanız büyük olasılıkla yanlış modelleme yapmışsınızdır
  - Çözüm kalıtım ve çok biçimliliktir; kalıtım uygun değilse State/Strategy.
- State ile Strategy arasındaki ince ama önemli fark
  - Sınıf şemaları aynıdır. State'te çalışma sırasında bir durumdan başkasına geçiş vardır; Strategy'de genelde bir kez seçilen strateji boyunca çalışılır.
- Move Method'da metot eski sınıftan silinmez
  - Eski yerde delegation kalır; böylece eski sınıfın arayüzü bozulmaz.
- Refactoring'in özü test kodunu değiştirmemektir
  - İş mantığı kodu değişip test kodu değişmiyorsa dış arayüz korunmuştur. Her adımdan sonra testler yeniden çalıştırılır ve yeşil çubuk görülür.
- Ad değiştirmek de bir refactoring eylemidir
  - Küçük olsa da anlaşılabilirliği artırdığı için küçümsenmemelidir.
- Geçici değişkeni kaldırmanın başarım bedeli önemsizdir
  - Bir müşteri 100 film bile kiralasa `getCharge`'ın 100 ya da 200 kez çağrılması fark edilmez.
- Her tasarım kalıbı bağlaşımı düşürmez
  - Kalıplar yeni sınıflar ve ilişkiler ekler; bağlaşımı bir noktaya kadar yükseltebilir. GoF kitabında bağlaşımı azaltan kalıplar olarak özellikle Facade, Chain of Responsibility, Command, Observer ve Mediator geçer.
- İyi tasarım, problemi hangi düzeyde parçalara ayırmak gerekiyorsa o düzeyde ayırır
  - Daha alt parçalara inmek gereksiz karmaşıklık yaratır.
- Hoca Eclipse'in otomatik refactoring seçeneklerine güvenmek yerine eylemleri elle yapmayı önerdi; yalnız ad değiştirmede Eclipse güvenilirdir.

Kısa Tekrar Notları

- Large Class → Extract Class, Extract Subclass, Duplicate Observed Data.
- Extract Class: ikinci işle ilgili üyeleri yeni sınıfa taşı; sahiplik ilişkisi kur.
- Extract Subclass: her zaman kullanılmayan üyeleri alt sınıfa taşı (Job Item → Labor Item).
- Duplicate Observed Data: GUI'deki iş verisini yeni sınıfa çıkar, aralarına Observer kur.
- Uyum ≈ gerçekleşen erişim / (metot sayısı × alan sayısı). Örnek: 10/24 ≈ 0,42 → 6/8 = 0,75 ve 4/4 = 1.
- Divergent Change: tek sınıf, çok metot. Shotgun Surgery: çok sınıf, az metot. Çözüm: Extract Class, Move Method, Move Field.
- Move Method: en çok ihtiyaç duyan sınıfa taşı, eski yerde delegation bırak.
- Primitive Obsession → Replace Data Value with Object, Replace Type Code with Class / Subclasses / State-Strategy, Replace Array with Object.
- Tür çalışma sırasında değişiyorsa State/Strategy, değişmiyorsa alt sınıflar.
- Feature Envy → Move Method, Extract Method (+ Move). Strategy ve Visitor Feature Envy sayılmaz.
- Fowler örneği sırası: Extract Method (`amountFor`) → Rename → Move Method (`Rental.getCharge`) → Replace Temp with Query (`thisAmount`) → `getBonus` → `getTotalCharge`/`getTotalBonus` → switch `Movie`'ye → bonus `Movie`'ye → `getMovieTitle` → (alternatif) `Price` stratejisi.
- Kara kutu: davranış; beyaz kutu: kod kapsamı.
- Karşıt kalıplar tasarım düzeyinde ve çok sınıflı; kod kusurları gerçekleme düzeyinde ve bir iki sınıflı.

Detaylı Açıklamalar

Uyum hesabı

Hoca Violet UML Editor'da bir şekil çizdi (ekrandan, 00:16:40): üstte `name`, `id`, `officePhoneCode`, `officePhoneNr` üye alanları, altta m1–m6 metotları; her kesikli ok bir metodun bir alana eriştiğini gösterir. Tam bağlı bir grafikte $6 \times 4 = 24$ erişim olurdu; çizilen 10 erişimle oran $10/24 \approx 0{,}42$'dir. Telefon alanları ve onlara erişen metotlar ayrı sınıfa alındıktan sonra birinci sınıfta $4 \times 2 = 8$ olası erişimin 6'sı (0,75), ikinci sınıfta $2 \times 2 = 4$ olası erişimin 4'ü (1,0) vardır. İki sınıfın uyumu da başlangıçtakinden yüksektir; Extract Class bu ölçüte göre de iyi bir iş yapmıştır.

- Ders sonrası not: Hoca bu oranı bir yerde "bağlaşım" diye andı (00:15:48). Hesaplanan büyüklük uyumdur (cohesion); konuşmanın geri kalanında hoca da "uyum" dedi.

Fowler örneği: kodun son hâli

`Rental` sınıfı (ekrandan, 02:40:00):

```
public double getCharge( ) {
    return _movie.getCharge( _daysRented );
}
public int getBonus( ) {
    return _movie.getBonus( _daysRented );
}
public String getMovieTitle( ) {
    return _movie.getTitle( );
}
```

`Movie` sınıfının ücret metodunun başı (ekrandan, 02:30:00):

```
public double getCharge( int daysRented ) {
    double result = 0;
    switch ( getPriceCode() ) {
    case Movie.REGULAR:
        result += 2;
        if (daysRented > 2)
            result += (daysRented - 2) * 1.5;
        break;
    case Movie.NEW_RELEASE:
    ...
```

`Customer`'a eklenen toplam metotları (ekrandan, 02:17:20):

```
public double getTotalCharge( ) {
    double result = 0;
    for( Rental each : _rentals )
        result += each.getCharge();
    return result;
}
public int getTotalBonus( ) {
    int result = 0;
    for( Rental each : _rentals )
        result += each.getBonus( );
    return result;
}
```

Ara adımda (`fowler_05`, ekrandan, 01:55:30) `Rental.getBonus()` şöyleydi; 8. adımda bu mantık `Movie.getBonus(int daysRented)`'e taşındı:

```
public int getBonus( ) {
    if ((getMovie().getPriceCode() == Movie.NEW_RELEASE)
            && getDaysRented() > 1)
        return 2;
```

```
    else
        return 1;
}
```

Toplamlar artık iki ayrı döngüde hesaplanıyor, yani kira listesi iki kez dolaşılıyor. Hoca, kişi başına kiralama sayısı küçük olduğu için bunun başarıma etkisinin hissedilmeyeceğini, kazanılan odaklılığın daha değerli olduğunu söyledi.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.